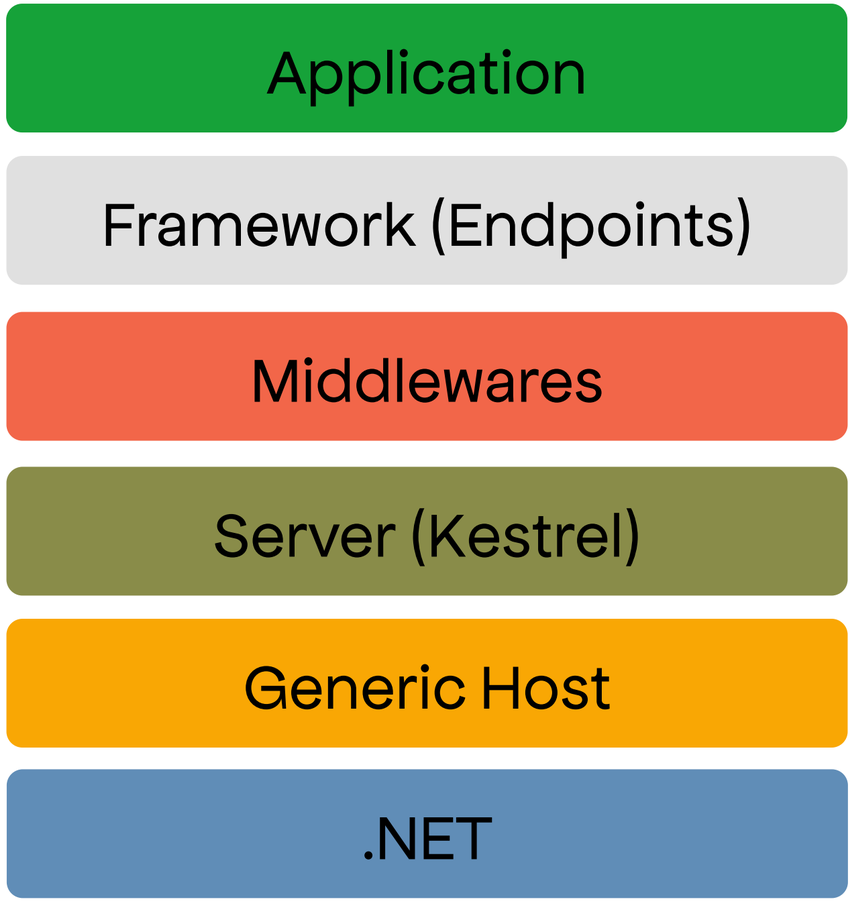
Application Architectures



DRIM

Application

Application



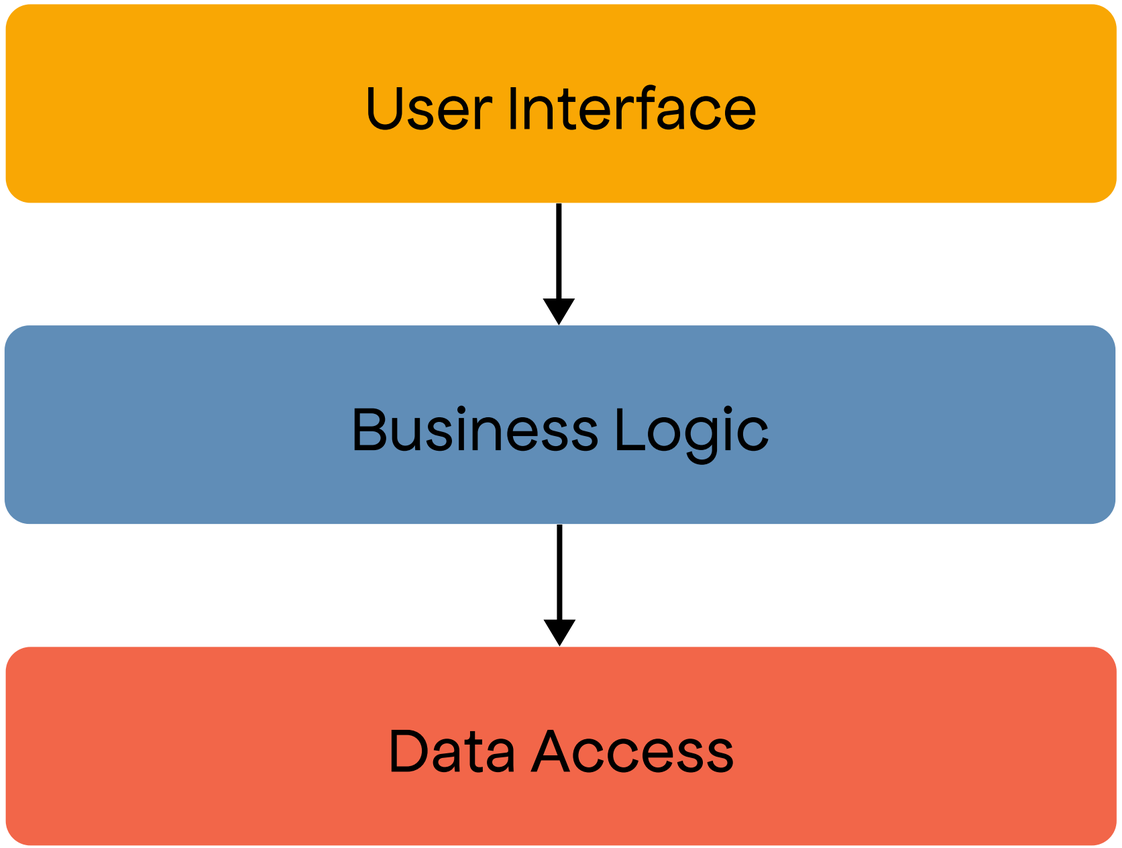
Application Architecture Goals

- Managing complexity (reducing accidental complexity)
- Application maintainability
- Accidental complexity vs evolvability
- Knowledge communication
- Managing obsolescence
- Reusability
- Development speed up

DRIM

Traditional Architectures

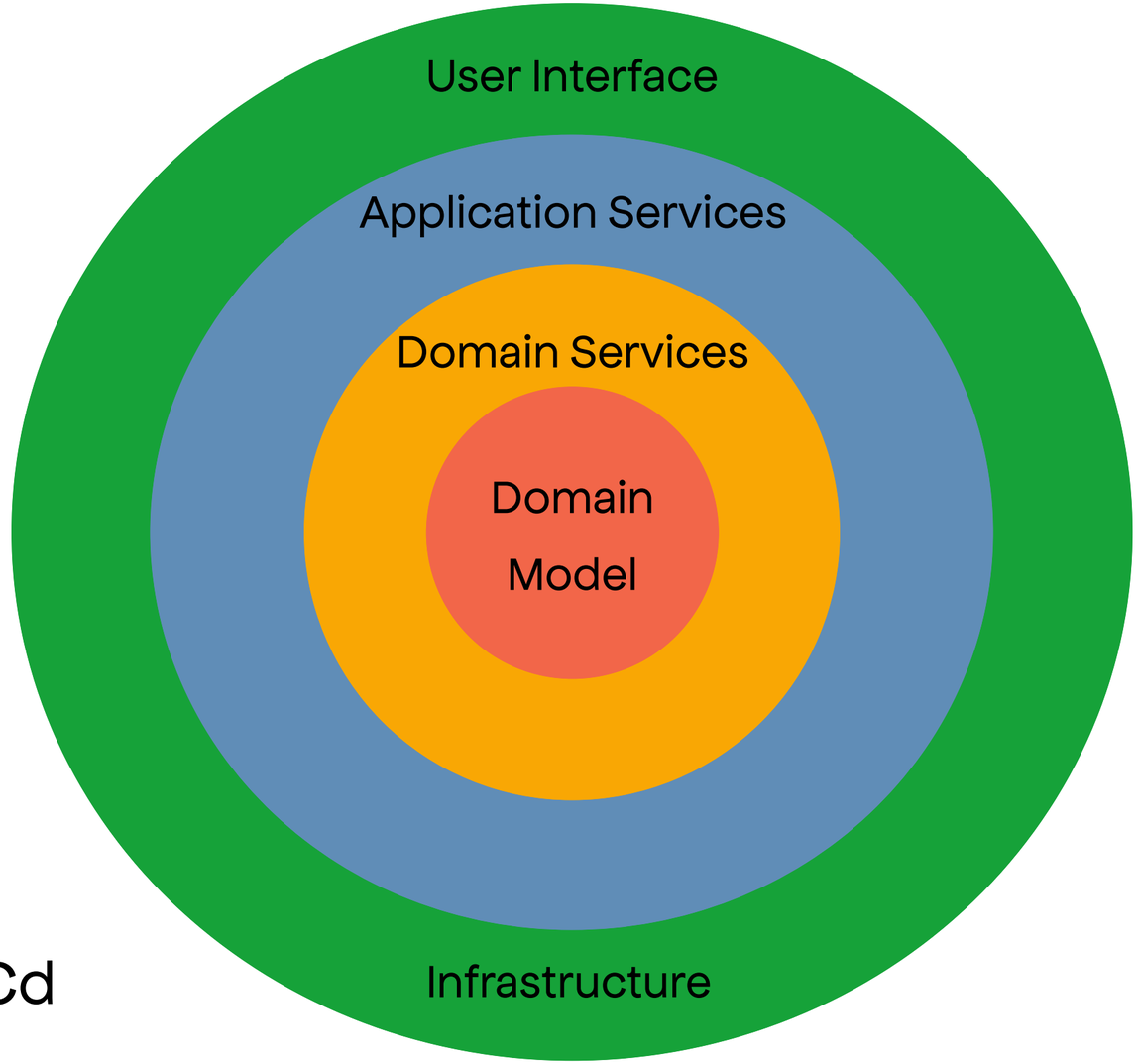
Layered Architecture



Layered Architecture

- Separation of concerns
- Abstraction and encapsulation
- Downward dependency direction
- Single responsibility
- Layer independence

Onion Architecture



https://bit.ly/3WmVjCd

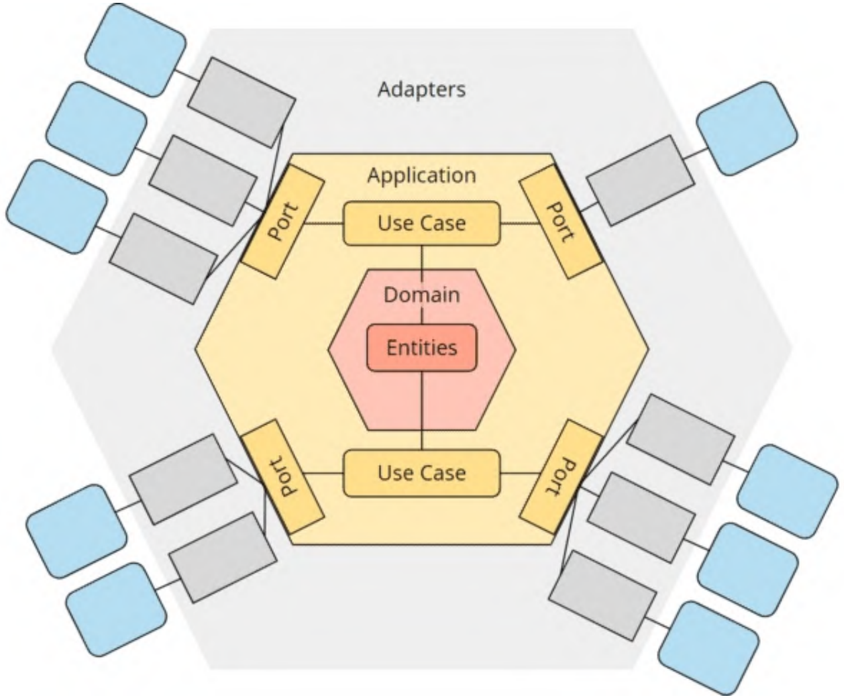
Onion Architecture

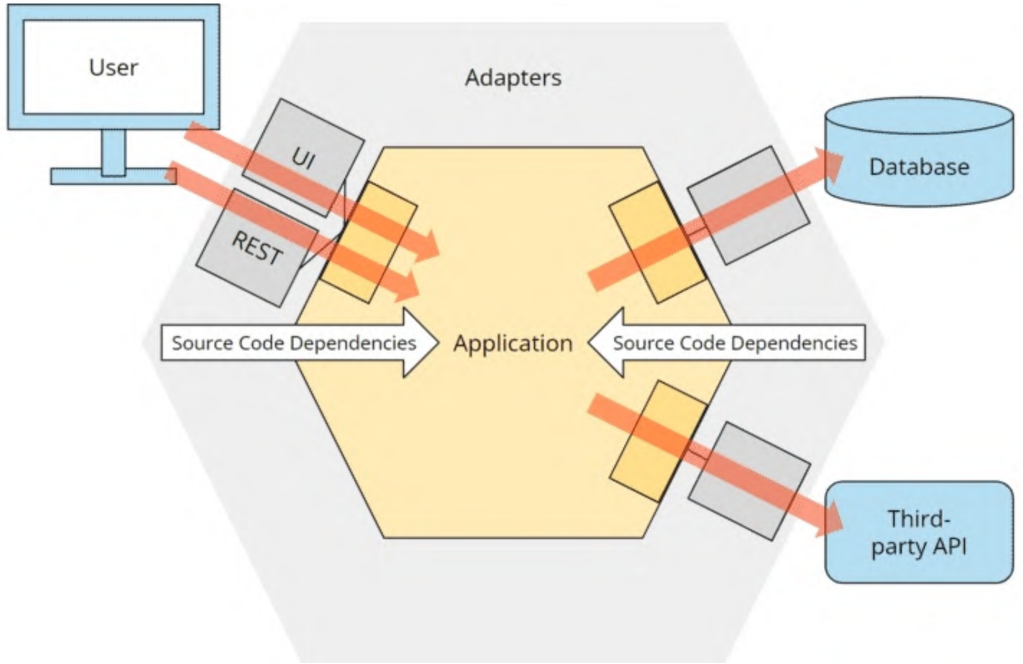
- **Domain Model + Domain Services.** This layer contains the core business logic and domain models. It represents the heart of the application and should be independent of any specific technology or infrastructure concerns
- **Application Services.** The application layer sits outside the domain layer and contains the application-specific logic. It orchestrates the interaction between the domain layer and the infrastructure layer, handling use cases and exposing services
- **Infrastructure.** This layer is responsible for interacting with external resources and frameworks, such as databases, file systems, or web services
- **User Interface.** This layer is responsible for presenting information to users and capturing their input. It includes user interfaces, such as web interfaces, desktop applications, or mobile apps

Onion Architecture

- The domain model is the truth of the organization
- The application is built around an independent object model
- Inner layers define interfaces. Outer layers implement interfaces
- The direction of the coupling is toward the center
- Deeper layers change less often
- All application core code can be compiled and run separately from the infrastructure
- Testability

Hexagonal Architecture



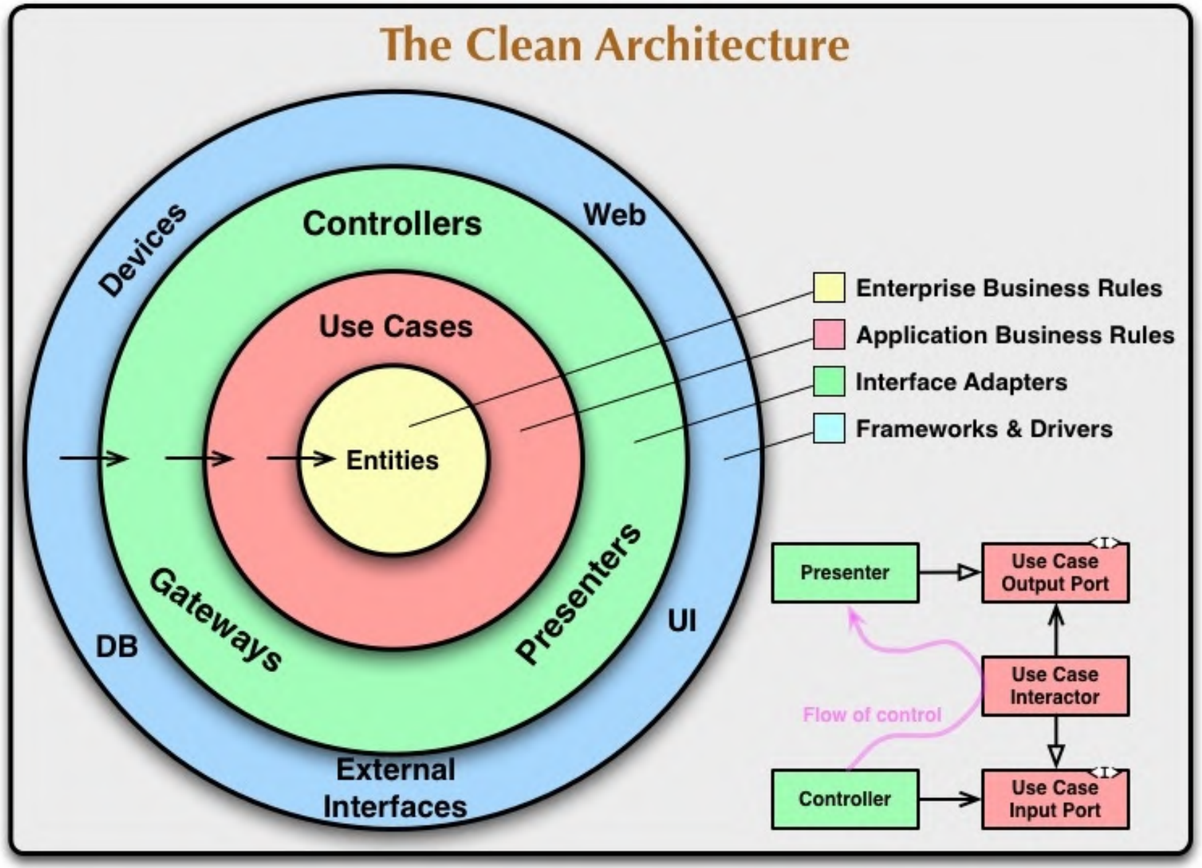


https://bit.ly/3ofxoIp

Hexagonal Architecture

- **Domain Model + Domain Services.** This layer contains the core business logic and domain models. It represents the heart of the application and should be independent of any specific technology or infrastructure concerns
- **Application Services.** The application layer sits outside the domain layer and contains the application-specific logic. It orchestrates the interaction between the domain layer and the infrastructure layer, handling use cases and exposing services
- **Ports.** Ports define the interfaces through which the core interacts with the outside world. They are the entry and exit points for data and control flow
- **Adapters.** Adapters implement the ports and connect the core with the external world. They are responsible for translating and transforming data between the core and the external systems, such as databases, user interfaces, or third-party services

Clean Architecture



https://bit.ly/3BNEqqS

Clean Architecture

Onion Architecture,

Hexagonal Architecture,

Ports and Adapters,

Clean Architecture

are

THE SAME THING

Domain-Driven Design

- Bounded contexts
- Aggregates
- Aggregate roots
- Entities
- Value objects
- Domain events

IMO, canonical DDD implementation in most cases leads to overly-engineered applications, though parts of it are useful

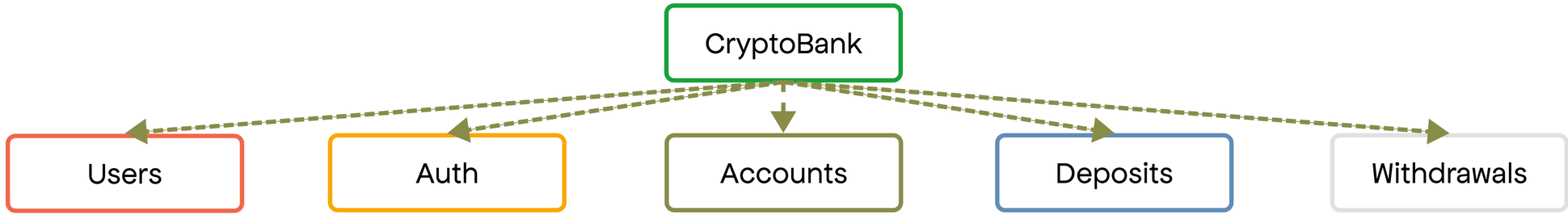
DRIM

Feature-based architecture

DRIM

CryptoBank Features

MODULES



FEATURES

- Register user
- Confirm Registration

- Authenticate
- Refresh Access Token
- Revoke Refresh Token

- Create Account
- Get Accounts
- Get Account Details
- Transfer
- Get Account Transfers

- Get Deposit Address
- Find New Deposits
- Update Deposit Confirmations
- Get Deposits

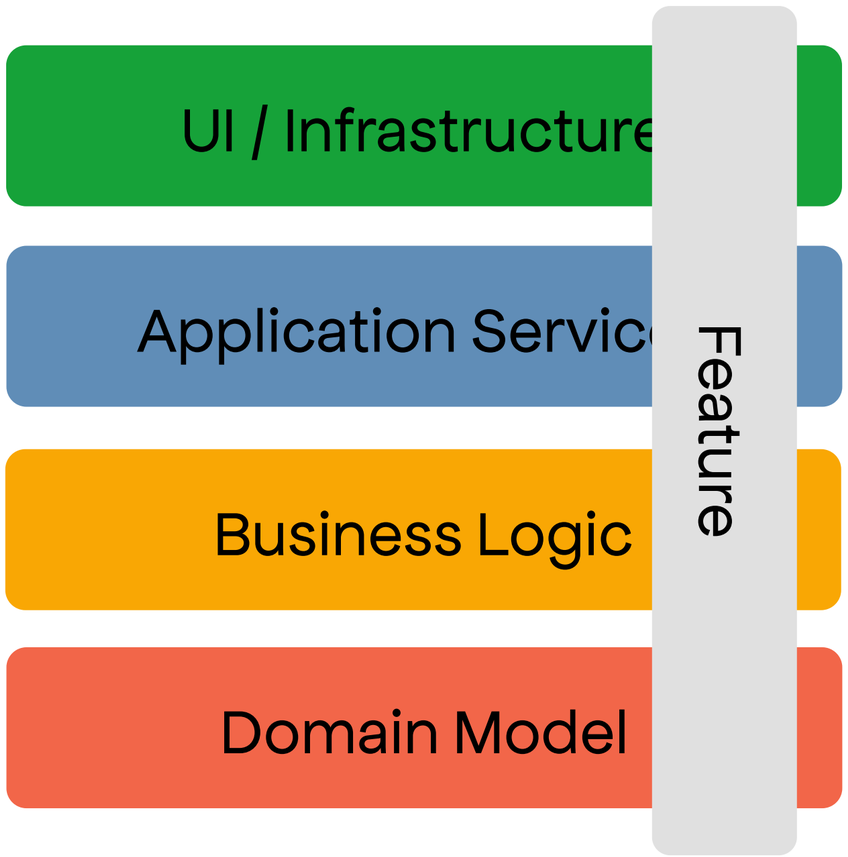
- Create Withdrawal
- Confirm Withdrawal
- Send New Withdrawals
- Update Withdrawal Confirmations
- Get Withdrawals

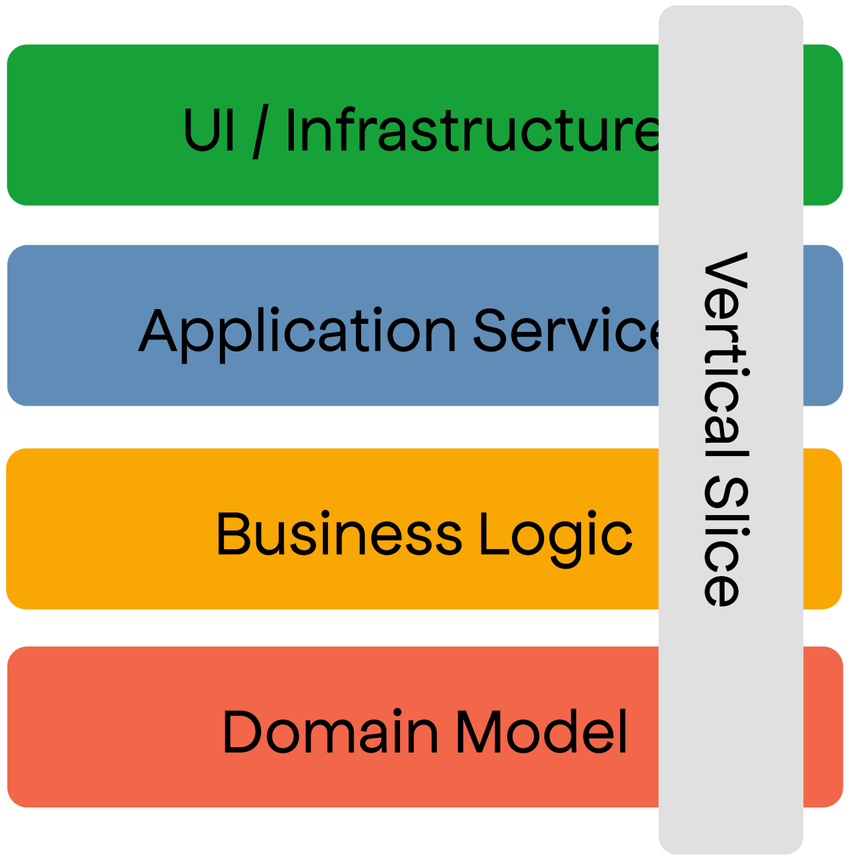
Screaming Architecture

The basic idea behind Screaming Architecture is to create an architecture that clearly reflects the business domain and requirements of the system. It aims to make the architecture "scream" the essential features, behaviors, and rules of the system, making it easily understandable and maintainable.

https://bit.ly/3MMmy5Q

Vertical Slices



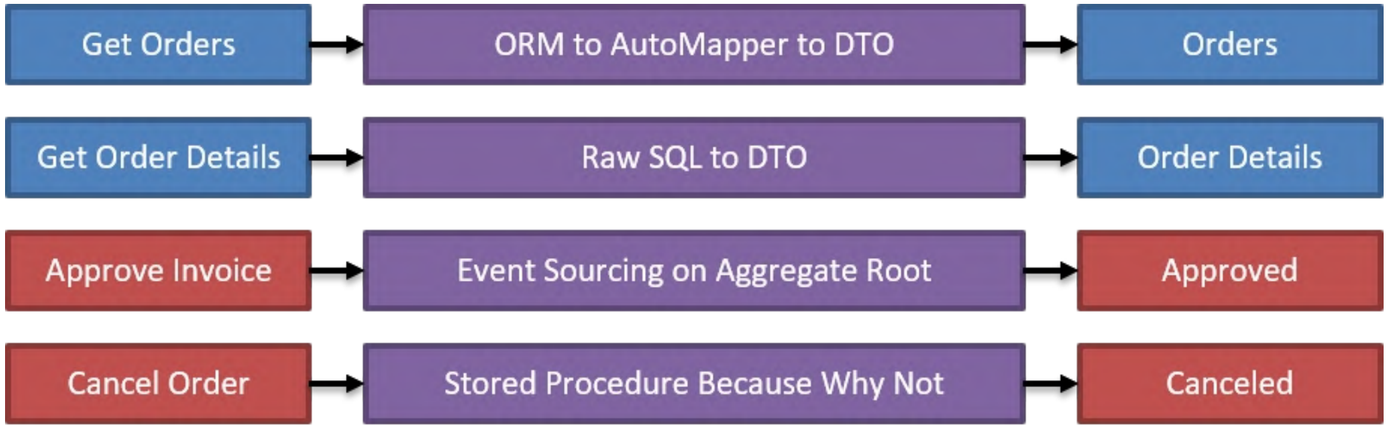


https://bit.ly/3BNpDMU

Vertical Slices

- Things that change together (features) belong together
- The project structure screams the domain
- Low coupling and high cohesion
- Cross-cutting concerns implemented easily
- Independent development and easier merging
- Easier navigation
- Easier testing

Vertical Slices



DRIM

Thank you! Questions?

